Research Update & Future Directions

Huy Khanh Tran

April 21, 2026

1 Objective

This report has 3 main objectives:

- Give details of the current research and experiments.
- State and explain current problems based on recent results.
- Request advice on how to proceed with the next phase of experiments.

The details of each block, experiment, and configuration are included in the Appendix (4) for reference.

2 Research Summary

The Core Claim: "Selecting objects for copy-paste augmentation based on empirically measured model failure scores produces better detection improvement than selecting objects randomly, when the dominant failure mode falls within the learnable regime of copy-paste augmentation."

The Supporting Claims

Claim 1 (Score Validity):
"ImageScorer reliably identifies objects and images where the model fails, as validated by its correlation with actual miss rate and false positive rate across two structurally different datasets."

Claim 2 (Augmentation Effectiveness):
"Score-guided copy-paste selection outperforms uniform random selection on the detection of learnable objects across multiple seeds and datasets."

Current Status:
We have completed the 2 blocks of experiments for Claim 1 and Claim 2 and analyzed the results. While Claim 1 is largely supported by strong correlations, Claim 2 requires more modification. Specifically, the score-guided copy-paste augmentation is currently showing a **decrease in overall performance metrics (AP)**. The details of this issue and our proposed fixes are described below.

3 Current Problems

3.1 Problem 1: Lower score stability on MinneApple when training without traditional augmentation

As seen in Appendix Exp 1.2 (Tables 3 and 4), the scoring system is highly stable across different seeds for Global Wheat (GWHD) and for MinneApple (MA) *with* traditional augmentation

(Pearson > 0.80). However, on MinneApple *without* traditional augmentation, the inter-seed correlation drops to ~ 0.67.

- **Hypothesis:** MinneApple contains heavily clustered, small objects. Without basic augmentations (like color jitter or flipping) to regularize the baseline model, the low-confidence predictions fluctuate wildly between random seeds. This causes the difficulty score for specific objects to become unstable.
- **My interpretation:** This may indicate that the scoring output is strongly dependent on the baseline model behavior. If the model changes, the score ranking also changes significantly, which makes stability a key requirement before using the score for downstream augmentation decisions.

3.2 Problem 2: Score-guided copy-paste augmentation decreases overall AP

As shown in Appendix Exp 2.1 (Table 7), applying the score-guided copy-paste reduced the overall COCO_AP from the baseline ($0.4472 \rightarrow 0.4280$).

- **The AP75 Drop:** While the AP50 actually *increased* ($0.7743 \rightarrow 0.7882$), the AP75 dropped significantly ($0.4637 \rightarrow 0.4288$). This indicates the model is finding more objects (loose boxes), but its localization precision has degraded. This is likely due to pasting hard, complex objects without blending or masks, causing the model to learn edge artifacts rather than true object boundaries.
- **The Large Object Drop:** We also saw a severe drop in AP_large ($0.7191 \rightarrow 0.6271$). Because we use exponential weighting (`weight_scale=3.5`) to sample hard objects/backgrounds, we may be over-saturating the images, occluding large objects, or creating highly out-of-distribution scenes.

4 Request for Advice

Based on the problems above, I have continued running many additional experiments to tune the augmentation hyperparameters.

Some settings gave better results for specific metrics, but most choices still lack a clear scientific reason for why that specific configuration should be preferred. In other words, I can find local improvements, but they are not consistently explainable.

When I look at the average behavior across runs/seeds, the overall performance improvement is still small or not clearly meaningful for this dataset.

Because of that, I would like your guidance on the research direction:

- Should I continue this line of work and spend more time on principled hyperparameter selection/ablation?
- Or should I pivot to another method/field that may have stronger potential and clearer scientific justification?

If you think I should continue this part, I would also appreciate advice on which of the following directions should be prioritized first:

1. **Scale the Parameter Search:** If we continue this direction, should I run a broader hyperparameter sweep to identify a robust and reproducible setting?
2. **Try a More Complex Copy-Paste Algorithm:** Should I move to a more advanced copy-paste pipeline (for example, better blending and placement constraints) to improve detection quality on agricultural datasets?

Appendix: Research Methodology & Configurations

This section contains the detailed methodology for reference.

A. Methodology

Part 1: Scoring System Design

1. Meaning:
The **scoring system** measures how difficult each object and image is for the model.

- Each **object** is scored based on how well it is detected.
- Each **image** is scored based on:
  - how difficult its objects are
  - how many false positives it contains

Important Notes:

- Object difficulty is computed using **low-confidence predictions**.
- False positives are computed using **optimal-threshold predictions**.

This separation ensures that we capture both:

- **model uncertainty** (via low-confidence predictions)
- **final prediction quality** (via optimal-threshold predictions)

---

2. Formulation
Object-level score (using low-confidence predictions):

$$S_{obj}(g) = \min_{p:\mathrm{IoU}(p,g)\geq\tau} [\alpha(1-\mathrm{conf}(p)) + \beta(1-\mathrm{IoU}(p,g))]$$

If no prediction matches:

$$S_{obj}(g) = \alpha + \beta$$

Image-level score (using optimal-threshold predictions):

$$S_{img}(I) = w_1 \cdot \overline{S_{obj}}(I) + w_2 \cdot \mathrm{FP\_rate}(I)$$

---

3. Why two different prediction sets?
Low-confidence predictions (for S_{obj}):

- Include many candidate detections (even weak ones)
- Allow the scorer to check:
  - Did the model *almost* detect the object?
  - Was there a nearby but low-confidence prediction?
- This captures **model awareness**, not just final decisions

Optimal-threshold predictions (for FP rate):

- Represent the model's **final output**
- Used in standard evaluation (precision/recall trade-off)
- Ensure false positives reflect **real deployment behavior**

---

4. Explanation of the formulas

Object score:

$$\alpha(1 - \text{conf}) \rightarrow \text{penalty for low confidence}$$

$$\beta(1 - \text{IoU}) \rightarrow \text{penalty for poor localization}$$

- High confidence + high IoU $\Rightarrow$ low score (easy object)
- Low confidence or low IoU $\Rightarrow$ high score (hard object)
- No matching prediction $\Rightarrow$ maximum penalty

Image score:

- $\overline{S_{obj}}$: average object difficulty
- FP_rate: fraction of predictions that are incorrect
- Many difficult objects $\Rightarrow$ high score
- Many false positives $\Rightarrow$ high score

---

5. Parameter table

Param	Role	Value/Mode	How it is chosen
α	Weight of confidence penalty	0.5	Set in config. Balances impact of confidence vs localization.
β	Weight of IoU penalty	0.5	Set in config. Typically equal to α for balanced scoring.
τ	IoU matching threshold	0.5	Standard detection threshold.
w_1	Weight of object difficulty	0.9	Chosen so that it balances with the false positive term.
w_2	Weight of false positives	adaptive	Chosen so that it balances with the object difficulty term.

---

6. Computation flow

1. Train baseline detection model.
2. Generate two prediction sets:
   - Low-confidence predictions (dense, exploratory)
   - Optimal-threshold predictions (final outputs)
3. For each image:
   (a) Compute S_{obj} for each object using low-confidence predictions

(b) Compute average object score

(c) Compute FP rate using optimal-threshold predictions

(d) Compute missed detection rate (for analysis only)

4. Select w_1 and w_2 to balance the contributions of object difficulty and false positives.
5. Compute final image score:

$$S_{img} = w_1 \cdot \overline{S_{obj}} + w_2 \cdot \text{FP\_rate}$$

---

7. Summary

- Low-confidence predictions → measure **how close the model was**
- Optimal-threshold predictions → measure **final errors**
- Combined score → robust measure of **difficulty**

Part 2. Copy-and-Paste Augmentation Pipeline

Key Idea:
This part generates additional training data by copying objects from labeled images and pasting them into other images. The goal is to **intentionally increase difficult training samples**, guided by the scoring system from Part 1. **Instead of random augmentation, this pipeline focuses on hard cases.** Objects and backgrounds are selected based on their **difficulty scores**, so the model sees more examples it previously struggled with.

How it is done (step-by-step):

1. Select a background image.
2. Select one or more objects to paste.
3. Determine how many objects to paste.
4. Apply small transformations (if needed).
5. Paste objects and generate new labels.

How backgrounds are chosen:

- Backgrounds are sampled from training images.
- Images with **higher difficulty scores are preferred**.
- Each background has a reuse limit to avoid repetition.

Why: Hard backgrounds already contain challenging context, so adding more objects creates even more valuable training samples.

How objects are chosen:

- Objects are ranked using **object difficulty scores**.
- Harder objects are sampled more frequently.
- Objects that are too large or incompatible with the background are removed.
- A fixed random seed ensures reproducibility.

Why: Hard objects represent model weaknesses, so repeating them improves learning efficiency.

How many objects are pasted:

- Controlled by: current object density, target density, and maximum paste limit.
- Sparse images are filled more, dense images are limited.

Why: Prevents unrealistic overcrowding while enriching sparse scenes.

How pasting is performed:

- Objects may be slightly **scaled or rotated**.
- Pasting is done with `blending_method = none`.
- No masks are used (bounding-box based).

Configuration Parameters

Parameter	Value	Function	Reason / Design Choice
dataset_ratio	MA: 0.3; GWHD: 0.3	Number of generated images	More augmentation for smaller datasets.
target_density	50	Desired object density	**Balances sparsity and overcrowding**.
max_paste	MA: 20; GWHD: 25	Maximum objects added	Prevents unrealistic scenes.
use_mask	false	Use segmentation masks	Keeps pipeline simple and efficient.
blending_method	none	Blending strategy	**Direct paste avoids instability and artifacts**.
max_bg_reuse	4	Background reuse limit	Avoids repeating the same image too often.
max_obj_reuse	4	Object reuse limit	Avoids overusing the same object.

Traditional augmentation settings used in baseline training

Method / Parameter	Value	Usage note
hsv_h	0.015	Hue jitter (small color shift).
hsv_s	0.7	Saturation jitter.
hsv_v	0.4	Brightness/value jitter.
degrees	10.0	Rotation range ($\pm 10°$).
translate	0.1	Translation as fraction of image size.
scale	0.4	Scale jitter.
shear	0.0	Disabled to avoid box distortion.
perspective	0.0	Disabled to avoid geometric artifacts.
fliplr	0.5	Horizontal flip probability.
flipud	0.0	Vertical flip disabled in current runs.
mosaic	0.0	Disabled.
mixup	0.0	Disabled.
copy_paste	0.0	Disabled in baseline training.
cutmix	0.0	Disabled.
close_mosaic	0	Mosaic closure step disabled.

B. Experiment Blocks

Block 1: Proving the Score is Valid

Exp 1.1: Score vs Miss Rate, False Positive Correlation. Target: High correlation.

Table 1. Score vs Miss Rate Correlation

Case	n_images	w_1	w_2	Pearson(score, miss_rate)	Spearman(score, miss_rate)
MA — w/ trad aug	536	0.9000	0.3450	0.6978	0.6810
MA — w/o trad aug	536	0.9000	0.9406	0.6788	0.6675
GWHD — w/ trad aug	2700	0.9000	0.7506	0.8158	0.7519
GWHD — w/o trad aug	2700	0.9000	0.6852	0.7180	0.6689

Table 2. Score vs False Positive Rate Correlation

Case	n_images	w_1	w_2	Pearson(score, fp_rate)	Spearman(score, fp_rate)
MA — w/ trad aug	536	0.9000	0.3450	0.6883	0.6498
MA — w/o trad aug	536	0.9000	0.9406	0.6983	0.6463
GWHD — w/ trad aug	2700	0.9000	0.7506	0.8180	0.6652
GWHD — w/o trad aug	2700	0.9000	0.6852	0.7157	0.6194

Interpretation: All four cases show strong positive correlation between score and both miss rate/false-positive rate. The strongest relationship appears in GWHD with traditional augmentation.

Exp 1.2: Score Stability Across Seeds. Target: Inter-seed has high correlation

Table 3. Inter-seed Pearson Correlation

Case	n_common_images	Pearson(123,456)	Pearson(123,789)	Pearson(456,789)	Mean pairwise
MA — w/ trad aug	536	0.8254	0.8088	0.7845	0.8062
MA — w/o trad aug	536	0.6738	0.6652	0.6794	0.6728
GWHD — w/ trad aug	2700	0.9048	0.8927	0.9029	0.9001
GWHD — w/o trad aug	2700	0.8648	0.8556	0.8697	0.8634

Table 4. Inter-seed Spearman Correlation

Case	n_common_images	Spearman(123,456)	Spearman(123,789)	Spearman(456,789)	Mean pairwise
MA — w/ trad aug	536	0.8287	0.8195	0.8000	0.8161
MA — w/o trad aug	536	0.6146	0.5833	0.6401	0.6127
GWHD — w/ trad aug	2700	0.8892	0.8922	0.8790	0.8868
GWHD — w/o trad aug	2700	0.8888	0.9013	0.8833	0.8911

Interpretation: Three cases satisfy the stability target strongly. MA without traditional augmentation is the only unstable case.

Exp 1.3: Score Domain Distribution (Global Wheat). Target: Hard domains should receive higher scores.

Table 5. Domain AP & Mean Score Ranking

Domain	n_images	AP	AP50	Mean Score	Difficulty Rank
rres_1	363	0.5724	0.9617	0.2596	1 (Easiest)
ethz_1	592	0.5499	0.9404	0.2911	3
inrae_1	146	0.5225	0.9544	0.2835	2
arvalis_3	457	0.4664	0.9182	0.3474	4
usask_1	153	0.4080	0.8737	0.3838	6
arvalis_1	827	0.4026	0.8898	0.3691	5
arvalis_2	162	0.3654	0.8130	0.4306	7 (Hardest)

Interpretation: Domains with lower AP have higher mean difficulty scores, proving the score captures domain-level difficulty effectively without labels.

Exp 1.4: Score vs Object Size (MinneApple). Target: Smaller objects should receive higher scores.

Table 6. MinneApple size-wise mean score and baseline AP

Size type	n_objects	Mean score	Baseline AP
small	16793	0.3777	0.1317
medium	6022	0.2216	0.5224
large	0	N/A	0.5796

Interpretation: Small objects have a higher mean difficulty score, perfectly matching the trend where baseline AP is lowest for small objects.

Block 2: Proving Augmentation Effectiveness
Exp 2.1: Four-Condition Comparison on MinneApple using YOLOv8s.

Table 7. MinneApple baseline vs copy-paste variants

Setting	COCO_AP	COCO_AP50	COCO_AP75	AP_small	AP_medium	AP_large
Initial baseline	0.4472	0.7743	0.4637	0.3075	0.5996	0.7191
Random copy-paste	0.4362	0.7695	0.4532	0.2903	0.5933	0.7158
Score-guided	0.4280	0.7882	0.4288	0.2979	0.5748	0.6271

Interpretation:

- Both variants reduce overall AP versus the baseline.
- Score-guided copy-paste is worst on overall AP and has the largest AP_large drop.
- AP50 increases, but AP75 decreases strongly, indicating lower localization precision.